

Affine Functions

Devi Prasad M

August 2026

1 Affine Functions and Affine Combinations

An *affine function* is a linear function followed by a translation. It can be written as

$$f(\mathbf{v}) = L(\mathbf{v}) + \mathbf{b},$$

where

- L is a linear function, and
- $\mathbf{b}$ is a fixed translation vector.

The linearity of L means that

$$L\left(\sum_{i=1}^k c_i \mathbf{v}_i\right) = \sum_{i=1}^k c_i L(\mathbf{v}_i).$$

A natural question is:

Does an affine function preserve linear combinations in the same way that a linear function does?

The answer is almost the same, but with one important restriction:

An affine function preserves affine combinations.

An *affine combination* is a weighted combination whose coefficients sum to 1.

1.1 Why Must the Coefficients Sum to 1?

Consider a weighted combination of vectors

$$\mathbf{v} = \sum_{i=1}^k c_i \mathbf{v}_i.$$

Applying the affine function gives

$$\begin{aligned} f\left(\sum_{i=1}^k c_i \mathbf{v}_i\right) &= L\left(\sum_{i=1}^k c_i \mathbf{v}_i\right) + \mathbf{b} \\ &= \sum_{i=1}^k c_i L(\mathbf{v}_i) + \mathbf{b}. \end{aligned}$$

Now consider the same weighted combination of the outputs:

$$\begin{aligned}\sum_{i=1}^k c_i f(\mathbf{v}_i) &= \sum_{i=1}^k c_i (L(\mathbf{v}_i) + \mathbf{b}) \\ &= \sum_{i=1}^k c_i L(\mathbf{v}_i) + \left(\sum_{i=1}^k c_i \right) \mathbf{b}.\end{aligned}$$

For the two expressions to be equal, we require

$$\mathbf{b} = \left(\sum_{i=1}^k c_i \right) \mathbf{b}.$$

For an arbitrary translation vector $\mathbf{b}$, this requires

$$\boxed{\sum_{i=1}^k c_i = 1.}$$

Therefore,

$$\boxed{f\left(\sum_{i=1}^k c_i \mathbf{v}_i\right) = \sum_{i=1}^k c_i f(\mathbf{v}_i) \quad \text{whenever} \quad \sum_{i=1}^k c_i = 1.}$$

This is the fundamental preservation property of an affine function.

1.2 Linear Combination vs. Affine Combination

It is important to distinguish between a linear combination and an affine combination.

A *linear combination* has the form

$$\sum_{i=1}^k c_i \mathbf{v}_i,$$

with no restriction on the coefficients.

An *affine combination* has the additional constraint

$$\boxed{\sum_{i=1}^k c_i = 1.}$$

The coefficients may be positive, zero, or negative.

For example,

$$0.4\mathbf{v}_1 + 0.6\mathbf{v}_2$$

is an affine combination because

$$0.4 + 0.6 = 1.$$

On the other hand,

$$2\mathbf{v}_1 + 3\mathbf{v}_2$$

is a linear combination, but not an affine combination, because

$$2 + 3 = 5 \neq 1.$$

If, in addition,

$$c_i \geq 0 \quad \text{for all } i,$$

then the affine combination is called a *convex combination*.
For example,

$$0.4\mathbf{v}_1 + 0.6\mathbf{v}_2$$

is both an affine combination and a convex combination.

2 A Concrete 2D Example

Consider the affine function

$$f(\mathbf{v}) = \mathbf{L}\mathbf{v} + \mathbf{b},$$

where

$$\mathbf{L} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ -1 \end{bmatrix}.$$

The matrix $\mathbf{L}$ doubles the first coordinate and triples the second coordinate.
The vector $\mathbf{b}$ then translates the result by

4 units in the x -direction

and

−1 unit in the y -direction.

Let

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

We choose the coefficients

$$c_1 = 0.4, \quad c_2 = 0.6.$$

Notice that

$$c_1 + c_2 = 0.4 + 0.6 = 1.$$

Therefore,

$$0.4\mathbf{v}_1 + 0.6\mathbf{v}_2$$

is an affine combination.

Geometrically, it lies on the line segment joining $\mathbf{v}_1$ and $\mathbf{v}_2$.

2.1 Method 1: Combine the Inputs First

First construct the affine combination:

$$\begin{aligned}\mathbf{v}_3 &= 0.4\mathbf{v}_1 + 0.6\mathbf{v}_2 \\ &= 0.4 \begin{bmatrix} 1 \\ 2 \end{bmatrix} + 0.6 \begin{bmatrix} 3 \\ 6 \end{bmatrix} \\ &= \begin{bmatrix} 0.4 \\ 0.8 \end{bmatrix} + \begin{bmatrix} 1.8 \\ 3.6 \end{bmatrix} \\ &= \begin{bmatrix} 2.2 \\ 4.4 \end{bmatrix}.\end{aligned}$$

Now apply the affine function:

$$\begin{aligned}f(\mathbf{v}_3) &= \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 2.2 \\ 4.4 \end{bmatrix} + \begin{bmatrix} 4 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 4.4 \\ 13.2 \end{bmatrix} + \begin{bmatrix} 4 \\ -1 \end{bmatrix} \\ &= \boxed{\begin{bmatrix} 8.4 \\ 12.2 \end{bmatrix}}.\end{aligned}$$

2.2 Method 2: Transform the Inputs First

Now let us reverse the order.

First transform $\mathbf{v}_1$:

$$\begin{aligned}f(\mathbf{v}_1) &= \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 4 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 2 \\ 6 \end{bmatrix} + \begin{bmatrix} 4 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 6 \\ 5 \end{bmatrix}.\end{aligned}$$

Similarly,

$$\begin{aligned}
f(\mathbf{v}_2) &= \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 3 \\ 6 \end{bmatrix} + \begin{bmatrix} 4 \\ -1 \end{bmatrix} \\
&= \begin{bmatrix} 6 \\ 18 \end{bmatrix} + \begin{bmatrix} 4 \\ -1 \end{bmatrix} \\
&= \begin{bmatrix} 10 \\ 17 \end{bmatrix}.
\end{aligned}$$

Now form the same affine combination of the outputs:

$$\begin{aligned}
0.4f(\mathbf{v}_1) + 0.6f(\mathbf{v}_2) &= 0.4 \begin{bmatrix} 6 \\ 5 \end{bmatrix} + 0.6 \begin{bmatrix} 10 \\ 17 \end{bmatrix} \\
&= \begin{bmatrix} 2.4 \\ 2.0 \end{bmatrix} + \begin{bmatrix} 6.0 \\ 10.2 \end{bmatrix} \\
&= \boxed{\begin{bmatrix} 8.4 \\ 12.2 \end{bmatrix}}.
\end{aligned}$$

Thus,

$$\boxed{f(0.4\mathbf{v}_1 + 0.6\mathbf{v}_2) = 0.4f(\mathbf{v}_1) + 0.6f(\mathbf{v}_2)}.$$

The two paths produce exactly the same result.

3 Why Does This Work?

The equality is not a coincidence arising from our particular numerical example. Consider the general case

$$f(\mathbf{v}) = L(\mathbf{v}) + \mathbf{b}.$$

Then

$$\begin{aligned}
f(c_1\mathbf{v}_1 + c_2\mathbf{v}_2) &= L(c_1\mathbf{v}_1 + c_2\mathbf{v}_2) + \mathbf{b} \\
&= c_1L(\mathbf{v}_1) + c_2L(\mathbf{v}_2) + \mathbf{b}.
\end{aligned}$$

On the other hand,

$$\begin{aligned}
c_1f(\mathbf{v}_1) + c_2f(\mathbf{v}_2) &= c_1(L(\mathbf{v}_1) + \mathbf{b}) + c_2(L(\mathbf{v}_2) + \mathbf{b}) \\
&= c_1L(\mathbf{v}_1) + c_2L(\mathbf{v}_2) + (c_1 + c_2)\mathbf{b}.
\end{aligned}$$

Therefore, the two expressions are equal exactly when

$$c_1 + c_2 = 1.$$

The crucial point is that the translation vector $\mathbf{b}$ appears

once

when we first combine the inputs, but it appears

$(c_1 + c_2)$ **times**

when we first transform the inputs and then combine the outputs.

The condition

$$c_1 + c_2 = 1$$

makes these two quantities identical.

4 The Key Idea

The difference between linear and affine functions can be summarized in two equations.

For a linear function,

$$L\left(\sum_i c_i \mathbf{v}_i\right) = \sum_i c_i L(\mathbf{v}_i)$$

with no restriction on the coefficients.

For an affine function,

$$f\left(\sum_i c_i \mathbf{v}_i\right) = \sum_i c_i f(\mathbf{v}_i) \quad \text{provided} \quad \sum_i c_i = 1.$$

The underlying idea is:

Linear maps preserve the origin; affine maps preserve affine relationships.

The condition

$$\sum_i c_i = 1$$

ensures that the translation component is counted exactly once.